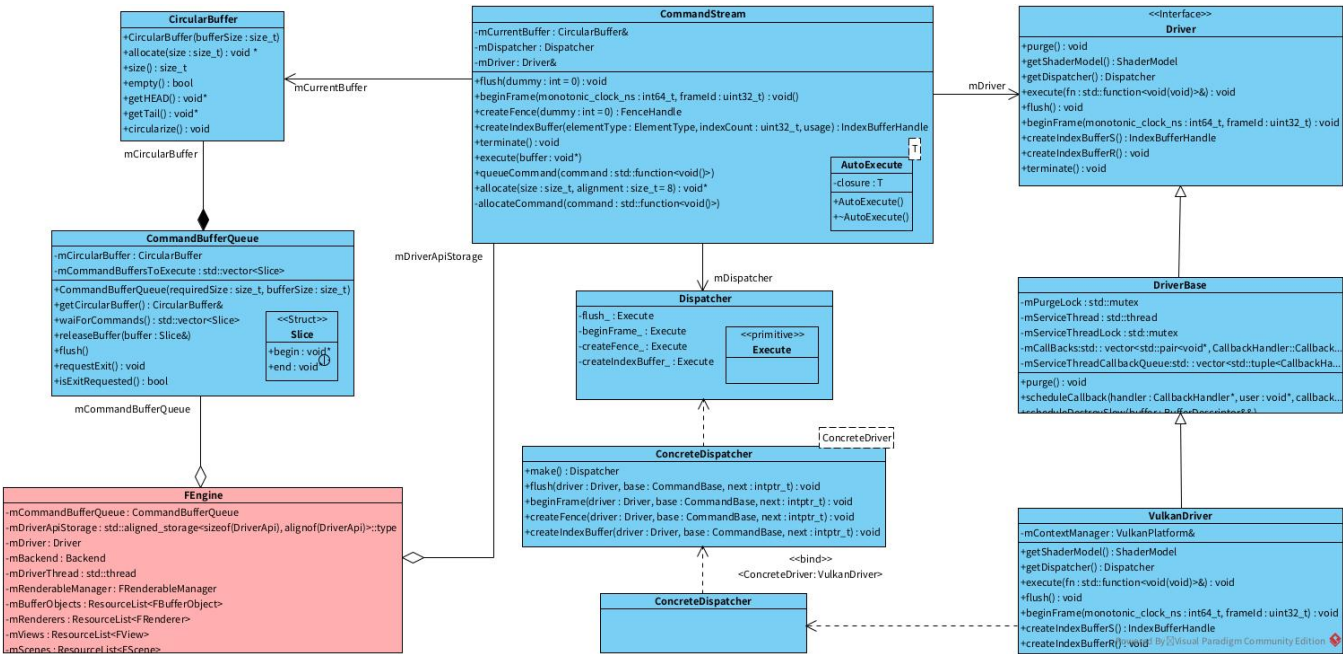


CommandStream指令创建过程

通过DriverAPI.inc的使用的文档，我们了解了DriverAPI.inc如何生成相关的代码的，然而文档中并没有针对每个部分及其调用关系做出解释。

下面从这个出发点来说明生成的代码之间是什么关系，以及如何调用的。

首先贴类图：



图中，我们可以看到，其中就包含了我们在DriverAPI.inc的使用中涉及到的所有类。

首先，简单说下他们的关系：

1. FEngine是引擎的具体实现类，包含了Engine的所有功能，实际上创建Engine的过程就是创建FEngine的过程。
2. 内聚合了CommandBufferQueue和CommandStream，CommandBufferQueue顾名思义，就是用来保存命令缓冲区的队列，CommandStream是命令流的抽象。
3. CommandBufferQueue内部组合了CircularBuffer，这也是CommandBufferQueue的核心，用来保存实际指令的数据结构。CircularBuffer抽象出了缓冲区的基本操作，CommandBufferQueue封装了CircularBuffer管理的接口和对指令处理相关的接口。
4. CommandStream关联了CircularBuffer和Driver，并组合了一个Dispatcher，CircularBuffer用来保存新命令，Dispatcher用来派发指令（也就是处理指令和具体实现的对应关系），Driver是一个接口，它抽象了各种渲染平台的基本功能，屏蔽了实现的差异。
5. Dispatcher内部定义了一个Execute的可执行类型，其实就是一个带有三个参数的无返回值的函数指针，而根据DriverAPI.inc的使用中，我们知道，里面保存了一系列可执行类型成员，也就是一系列函数指针。
6. 我们看到上面有两个ConcreteDispatcher，上面的带一个模板参数，可以看作是Dispatcher的工厂，用来根据模板参数创建具体的Dispatcher，下面的，则是在VulkanDriver部分针对上面ConcreteDispatcher显式实例化的类，我们这里模板参数使用的是VulkanDriver。
7. Driver有一个子类DriverBase，其主要加入了回调相关的功能，它的子类是真正实现平台化调用的具体内容，这里以VulkanDriver为参考，类似的还有OpenGLDriver、MetalDriver等。

下面，我们以createFence接口来讲解代码是怎么调用的。

通过阅读代码，我们可以找到，createFence接口实际上是被封装在了Engine类中：

```
class UTILS_PUBLIC Engine {
public:
    using Platform = backend::Platform;
    using Backend = backend::Backend;
    using DriverConfig = backend::Platform::DriverConfig;

    Fence* createFence() noexcept;
    ...
};

Fence* Engine::createFence() noexcept {
    return downcast(this)->createFence(FFence::Type::SOFT);
}
```

从函数实现看，实际上是将Engine对象向下转型为FEngine，并调用其createFence，而Engine的实现类其实就是FEngine，基本上绝大多数Engine的接口都是这样调用的。之后遇到的所有Engine接口我们都可以优先第一时间去FEngine去搜索实现。

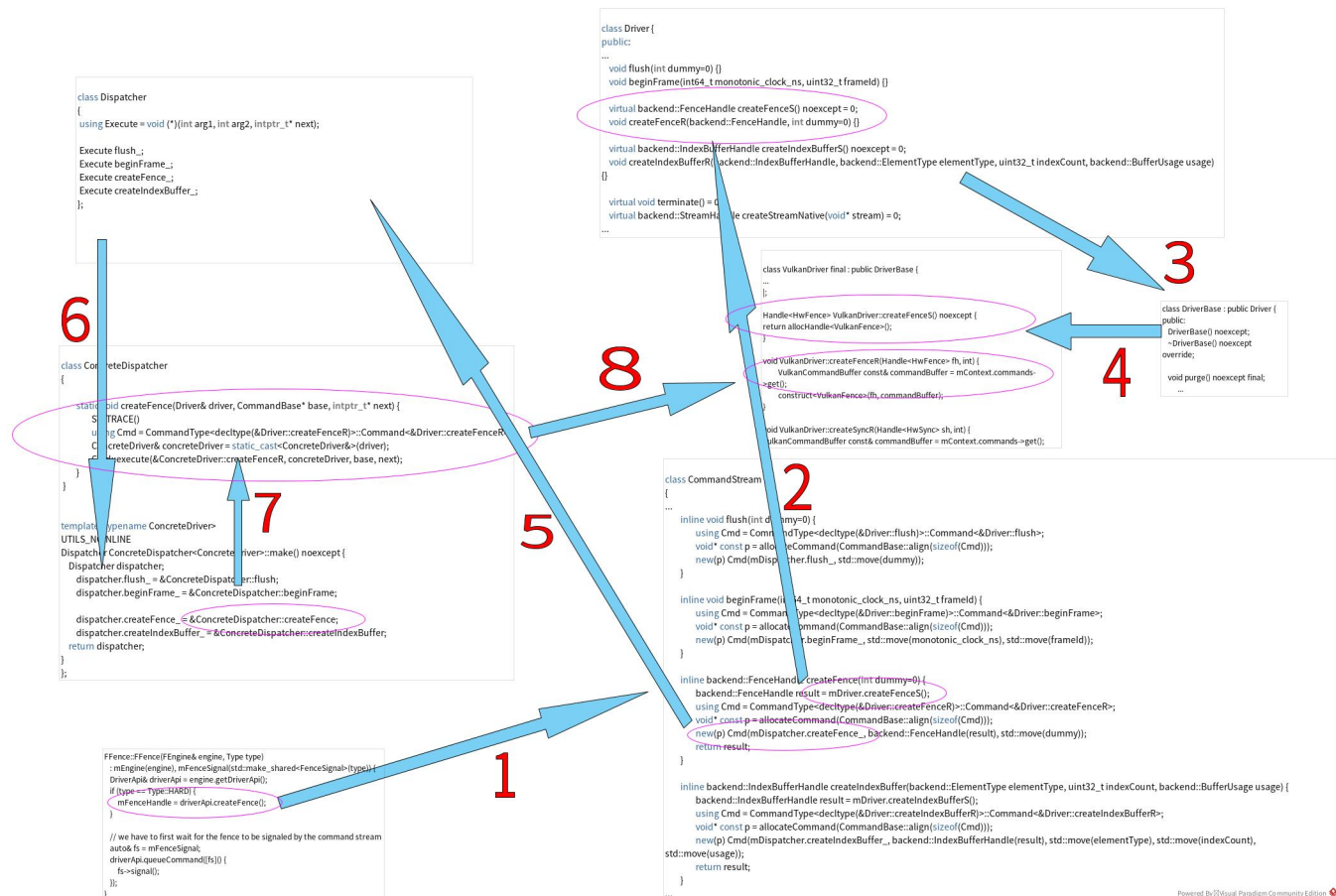
所以我们直接去FEngine中去看：

```
class FEngine : public Engine {
public:
    FFence* createFence(FFence::Type type) noexcept;
    ...
};

FFence* FEngine::createFence(FFence::Type type) noexcept {
    FFence* p = mHeapAllocator.make<FFence>(*this, type);
    if (p) {
        std::lock_guard guard(mFenceListLock);
        mFences.insert(p);
    }
    return p;
}
```

我们可以看到，FEngine的实现很简单，通过Filament Allocator所述的HeapAllocator来分配FFence的对象空间，并将FFence对象通过placement new来创建在这个位置。并将创建的FFence对象指针存入ResourceList所讲述的资源列表中进行管理。

也就是说，我们接口实现要接着往FFence的构造函数中去看看在代码前，我们先将调用示意图来贴出，方便我们更直观得理解调用过程：



下面我们继续看FFence的构造函数：

```

FFence::FFence(FEngine& engine, Type type)
: mEngine(engine), mFenceSignal(std::make_shared<FenceSignal>(type)) {
    DriverApi& driverApi = engine.getDriverApi();
    if (type == Type::HARD) {
        mFenceHandle = driverApi.createFence();
    }

    // we have to first wait for the fence to be signaled by the command stream
    auto& fs = mFenceSignal;
    driverApi.queueCommand([fs]() {
        fs->signal();
    });
}

```

我们看到，其中通过`engine.getDriverApi()`来获取到`CommandStream`的引用（在`DriverApiForward.h`中，`DriverApi`被定义为了`CommandStream`），并通过这个引用来调用`createFence`接口：

```
mFenceHandle = driverApi.createFence();
```

这就是图中步骤1所示的过程（这里的步骤是我们分析过程的步骤，不一定是严格的代码实际执行的过程）。

对我们来说，其实就是直接去找`CommandStream`的`createFence`接口即可。而在[DriverAPI.inc的使用](#)中，我们知道了`CommandStream`的实际内容，我们只看`createFence`接口实现：

```

class CommandStream
{
    ...

    inline backend::FenceHandle createFence(int dummy=0) {
        backend::FenceHandle result = mDriver.createFenceS();
        using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.createFence_, backend::FenceHandle(result), std::move(dummy));
        return result;
    }
    ...
};

```

我们看到，首先通过`mDriver.createFenceS()`调用返回了一个`FenceHandle`，而这里的`mDriver`其实就是前面说的`CommandStream`里面关联的`Driver`的引用。这就会调用到`Driver`的`createFenceS()`，也就是图中步骤2的过程。

通过[DriverAPI.inc的使用](#)我们知道，`Driver`实际内容为：

```

class Driver {
public:
    ...
    virtual backend::FenceHandle createFenceS() noexcept = 0;
    void createFenceR(backend::FenceHandle, int dummy=0) {}
    ...
};

```

而我们同样知道，这是一个纯虚函数，也就是说`Driver`实际上是一个接口，无法实例化。通过`Driver`的引用来调用虚函数，则会产生多态的效果。也就是会调用到它的派生类的实现。

而`DriverBase`是基于`Driver`继承而来，所以我们可以通过步骤3来找到`DriverBase`定义：

```

class DriverBase : public Driver {
public:
    DriverBase() noexcept;
    ~DriverBase() noexcept override;

    void purge() noexcept final;

    // -----
    // Privates
    // -----

protected:
    class CallbackDataDetails;

    // Helpers...
    struct CallbackData {
        CallbackData(CallbackData const &) = delete;
        CallbackData(CallbackData&&) = delete;
        CallbackData& operator=(CallbackData const &) = delete;
        CallbackData& operator=(CallbackData&&) = delete;
        void* storage[8] = {};
        static CallbackData* obtain(DriverBase* allocator);
        static void release(CallbackData* data);
    protected:
        CallbackData() = default;
    };

    template<typename T>
    void scheduleCallback(CallbackHandler* handler, T& functor) {
        CallbackData* data = CallbackData::obtain(this);
        static_assert(sizeof(T) <= sizeof(data->storage), "functor too large");
        new(data->storage) T(std::forward<T>(functor));
        scheduleCallback(handler, data, (CallbackHandler::Callback)[](void* data) {
            CallbackData* details = static_cast<CallbackData*>(data);
            void* user = details->storage;
            T& functor = *static_cast<T*>(user);
            functor();
            functor.~T();
            CallbackData::release(details);
        });
    }

    void scheduleCallback(CallbackHandler* handler, void* user, CallbackHandler::Callback callback);

    inline void scheduleDestroy(BufferDescriptor&& buffer) noexcept {
        if (buffer.hasCallback()) {
            scheduleDestroySlow(std::move(buffer));
        }
    }

    void scheduleDestroySlow(BufferDescriptor&& buffer) noexcept;

    void scheduleRelease(AcquiredImage const& image) noexcept;

    void debugCommandBegin(CommandStream* cmds, bool synchronous, const char* methodName) noexcept override;
    void debugCommandEnd(CommandStream* cmds, bool synchronous, const char* methodName) noexcept override;

private:
    std::mutex mPurgeLock;
    std::vector<std::pair<void*, CallbackHandler::Callback>> mCallbacks;

    std::thread mServiceThread;
    std::mutex mServiceThreadLock;
    std::condition_variable mServiceThreadCondition;
    std::vector<std::tuple<CallbackHandler*, CallbackHandler::Callback, void*>> mServiceThreadCallbackQueue;
    bool mExitRequested = false;
};

```

我们看到，其中，并没有createFenceS相关定义。也就是说，我们还需要再往下一级派生类来查找。最终，我们在众多代码中找到了名为VulkanDriver的子类，也就是步骤4所示过程。

而通过DriverAPI.inc的使用中的分析，我们VulkanDriver内的实际内容为：

```
class VulkanDriver final : public DriverBase {
public:
    static Driver* create(VulkanPlatform* platform,
        const char* const* ppEnabledExtensions, uint32_t enabledExtensionCount, const Platform::
DriverConfig& driverConfig) noexcept;
    Handle<HwFence> createFenceS();
    void createFenceR(Handle<HwFence> fh, int);
    ...
};

void VulkanDriver::createFenceR(Handle<HwFence> fh, int) {
    VulkanCommandBuffer const& commandBuffer = mContext.commands->get();
    construct<VulkanFence>(fh, commandBuffer);
}

Handle<HwFence> VulkanDriver::createFenceS() noexcept {
    return allocHandle<VulkanFence>();
}
```

其中声明和定义了结尾带 ‘S’ 和 ‘R’ 的createFence函数。

所以，Driver类的createFenceS实际调用的就是VulkanDriver的createFenceS。

再回到CommandStream的createFence，我们刚刚分析了步骤2的分支，下面继续看代码。

我们看到，后面通过placement new创建了Cmd的对象，并将其存放在分配的p的地址空间。

其中，Cmd定义我们参考CommandStream中：

```

class CommandStream
{
...
    inline void flush(int dummy=0) {
        using Cmd = CommandType<decltype(&Driver::flush)>::Command<&Driver::flush>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.flush_, std::move(dummy));
    }

    inline void beginFrame(int64_t monotonic_clock_ns, uint32_t frameId) {
        using Cmd = CommandType<decltype(&Driver::beginFrame)>::Command<&Driver::beginFrame>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.beginFrame_, std::move(monotonic_clock_ns), std::move(frameId));
    }

    inline backend::FenceHandle createFence(int dummy=0) {
        backend::FenceHandle result = mDriver.createFenceS();
        using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.createFence_, backend::FenceHandle(result), std::move(dummy));
        return result;
    }

    inline backend::IndexBufferHandle createIndexBuffer(backend::ElementType elementType, uint32_t
indexCount, backend::BufferUsage usage) {
        backend::IndexBufferHandle result = mDriver.createIndexBufferS();
        using Cmd = CommandType<decltype(&Driver::createIndexBufferR)>::Command<&Driver::
createIndexBufferR>;
        void* const p = allocateCommand(CommandBase::align(sizeof(Cmd)));
        new(p) Cmd(mDispatcher.createIndexBuffer_, backend::IndexBufferHandle(result), std::move
(elementType), std::move(indexCount), std::move(usage));
        return result;
    }
...
};

```

我们看到，Cmd其实是用Driver中每个接口的不同函数指针来实例化的模板类型声明（实例化模板类CommandType内部的Command类）：

```

using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;

```

查看CommandType和Command声明：

```

template<typename... ARGS>
struct CommandType<void (Driver::*)(ARGS...)> {

    template<void(Driver::*)(ARGS...)>
    class Command : public CommandBase {
    ...
    public:
        template<typename M, typename D>
        ...
        template<typename... A>
        inline explicit constexpr Command(Execute execute, A&& ... args)
            : CommandBase(execute), mArgs(std::forward<A>(args)...) {
        }

        // placement new declared as "throw" to avoid the compiler's null-check
        inline void* operator new(std::size_t size, void* ptr) {
            assert_invariant(ptr);
            return ptr;
        }
    };
};

```

我们看构造函数，它接收两个参数，第一个就是传入的可执行类型对象，第二个，就是要传给它的一个可变参数列表。通过CommandStream内部的函数定义，可变参数列表默认第一个参数就是Driver中以‘S’结尾的接口返回的Handle对象。其余参数就根据实际情况，每个Cmd构造时候都不一定相同。

通过分析每个Cmd，其中的可执行对象，实际上都是Dispatcher对象mDispatcher内部的一个成员。

Dispatcher定义参考[DriverAPI. inc的使用](#)中分析：

```
class Dispatcher {
public:
    using Execute = void (*)(Driver& driver, CommandBase* self, intptr_t* next);
    Execute flush_;
    Execute beginFrame_;
    Execute createFence_;
    Execute createIndexBuffer_;
};
```

实际上就是保存了一系列函数指针的类。如步骤5。

而mDispatcher，就是上面说的CommandStream中组合的Dispatcher对象。说到组合，那它的创建过程肯定是在CommandStream中：

```
CommandStream::CommandStream(Driver& driver, CircularBuffer& buffer) noexcept
    : mDriver(driver),
      mCurrentBuffer(buffer),
      mDispatcher(driver.getDispatcher())
    ...
{
    ...
}
```

这里通过初始化列表将driver.getDispatcher()返回的对象用来构造mDispatcher。driver对我们来说就是前面说的VulkanDriver、OpenGLDriver、MetalDriver其中之一，我们调试环境为VulkanDriver，那么：

```
Dispatcher VulkanDriver::getDispatcher() const noexcept {
    return ConcreteDispatcher<VulkanDriver>::make();
}
```

我们看到它实际上就是调用了ConcreteDispatcher<VulkanDriver>的make()。

还是要参考[DriverAPI. inc的使用](#)中的分析，其中对应的就是ConcreteDispatcher的第二部分：

```
template<typename ConcreteDriver>
UTILS_NOINLINE
Dispatcher ConcreteDispatcher<ConcreteDriver>::make() noexcept {
    Dispatcher dispatcher;
    dispatcher.flush_ = &ConcreteDispatcher::flush;
    dispatcher.beginFrame_ = &ConcreteDispatcher::beginFrame;

    dispatcher.createFence_ = &ConcreteDispatcher::createFence;
    dispatcher.createIndexBuffer_ = &ConcreteDispatcher::createIndexBuffer;
    return dispatcher;
}
```

这个过程参考上图步骤6。

可以看到，make调用过程，实际上就是给Dispatcher对象内部各个成员赋值的过程，而真正赋值上去的，就是[DriverAPI. inc的使用](#)中描述的ConcreteDispatcher的第二部分：

```

template<typename ConcreteDriver>
class ConcreteDispatcher
{
    static void flush(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::flush)>::Command<&Driver::flush>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::flush, concreteDriver, base, next);
    }

    static void beginFrame(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::beginFrame)>::Command<&Driver::beginFrame>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::beginFrame, concreteDriver, base, next);
    }

    static void createFence(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::createFenceR)>::Command<&Driver::createFenceR>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::createFenceR, concreteDriver, base, next);
    }

    static void createIndexBuffer(Driver& driver, CommandBase* base, intptr_t* next) {
        SYSTRACE()
        using Cmd = CommandType<decltype(&Driver::createIndexBufferR)>::Command<&Driver::
createIndexBufferR>;
        ConcreteDriver& concreteDriver = static_cast<ConcreteDriver&>(driver);
        Cmd::execute(&ConcreteDriver::createIndexBufferR, concreteDriver, base, next);
    }
};

```

参考上图步骤7。

这里面我们可以看到，由步骤5位置的Cmd创建过程实际结果，其实就是将ConcreteDispatcher<VulkanDriver>的函数指针保存在了Cmd中。而ConcreteDispatcher<VulkanDriver>中各个函数，最终会调用Cmd的execute()，并调用到ConcreteDriver中的具体以‘R’结尾的函数中。参考步骤8。

那么这个函数是如何被执行的呢？参考后续文档[CommandStream指令执行过程](#)。